

КМИТ, 7 клас, Маврова-Ненкова



Основни типове данни в Python. Булеви.

Как компютрите разбират "Истина" и "Лъжа"

Какво е булев тип (bool)?

Булевият тип има само две възможни състояния: **включено** или **изключено**, **да** или **не**.



True

```
is_open = True
```



Внимание! В Python булевите стойности задължително започват с главна буква! (Не "true", а "True").



False

```
is_open = False
```


Оператори за сравнение

Operator Symbol		Meaning
<	→	по-малко
<=	→	по-малко или равно
>	→	по-голямо
>=	→	по-голямо или равно
==	→	равно
!=	→	различно (не е равно)



Syntax Alert

Правилен ред! При <= и >=, знаците се пишат точно в този ред. Компютърът няма да разбере =<.



Сравняване на числа в действие

```
Code Editor

a = 5
b = -10
c = 15

print(c > a)
```

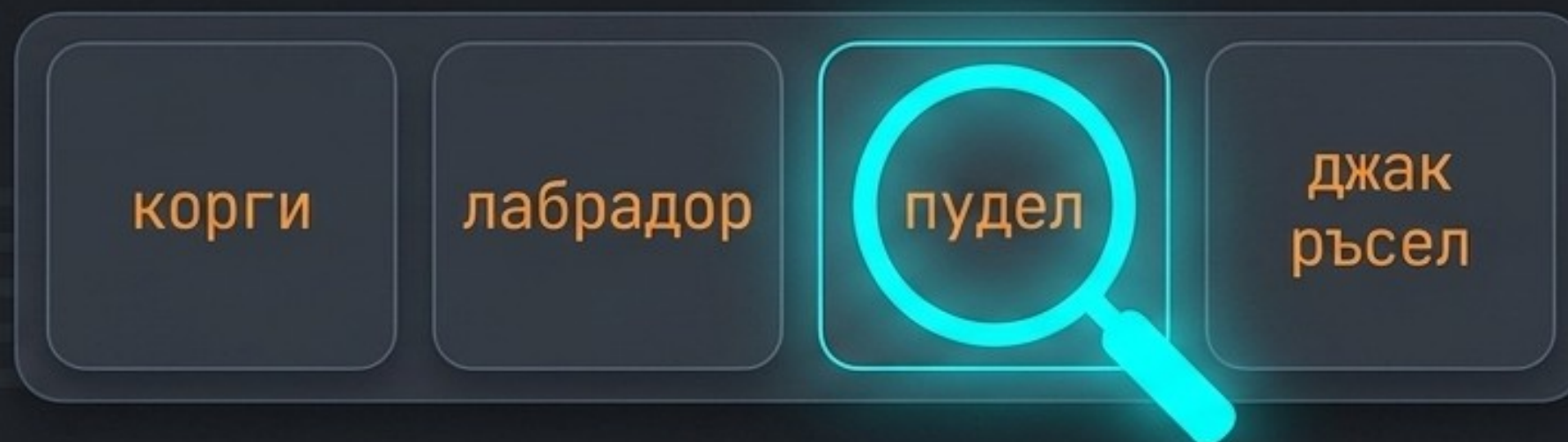
Компютърът пита:
"15 по-голямо ли е
от 5?"

```
Terminal Output

> True
```


Търсене в списъци с оператора **in**

Операторът **in** проверява принадлежност — дали даден елемент се намира вътре в списък.



```
Python Code  
  
dogs = ['корги', 'лабрадор', 'пудел', 'джак ръсел']  
x = 'пудел' in dogs
```

✓ Резултатът е True, защото пуделът присъства в списъка!



Търсене в текст с оператора in

Можем да използваме in, за да проверим дали един текст се съдържа в друг. Компютърът търси точно съвпадение на буквите!

"краставици и **дом**ати"

```
print("дом" in "краставици и домати")
```



Ще изведе **True**! Въпреки че "дом" (къща) няма нищо общо с домите, самата последователност от букви се съдържа там.



Тайната самоличност на булевите данни

В Python булевите данни могат да участват в истински математически задачи! Зад кулисите компютърът ги замества с числа:

True 1

True = 1

False 0

False = 0

Математика с Булеви стойности (Стъпка по стъпка)

```
x = (10 > 4) + 5
```

(10 > 4) се оценява като...

```
x = True + 5
```

True се превръща в 1...

```
x = 1 + 5
```

Резултат: 6



Логически операции

Както събираме числа, така можем да комбинираме и булеви стойности. Запознайте се с трите основни логически операции в Python:



and

(Логическо И)



or

(Логическо ИЛИ)



not

(Логическо ОТРИЦАНИЕ)

Операторът and (Логическо И)

Резултатът е True САМО АКО И ДВЕТЕ стойности са True.
В противен случай е False.



True and True → ☒ True

True and False → ☐ False

False and False → ☐ False

Операторът **or** (Логическо ИЛИ)

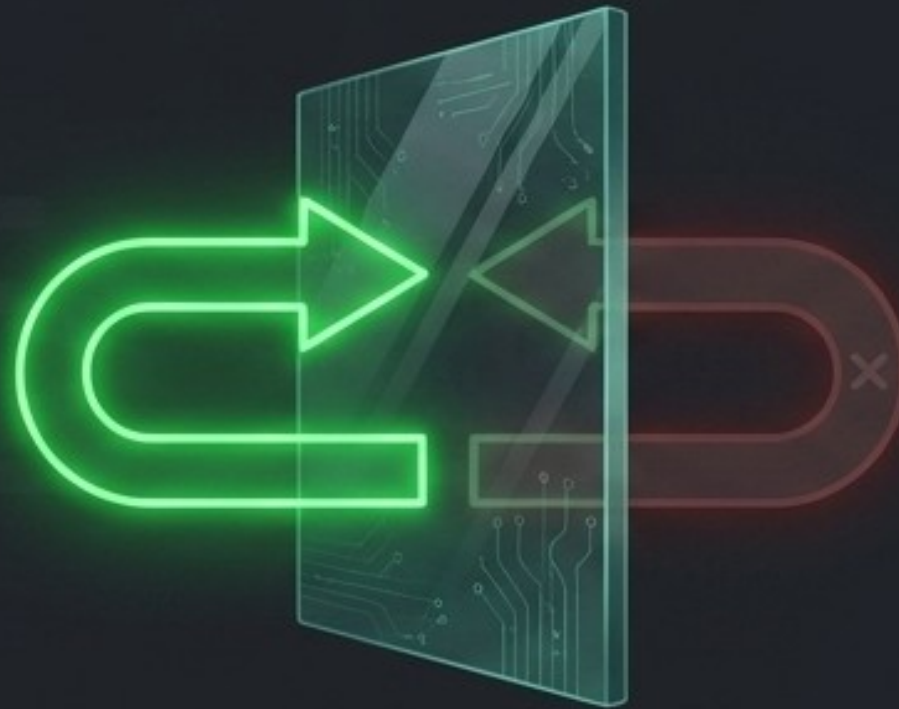
Резултатът е True, ако **ПОНЕ ЕДНАТА** от стойностите е True.



True or True	→	✓ True
True or False	→	✓ True
False or False	→	✗ False

Операторът not (Логическо ОТРИЦАНИЕ)

Това е **най-простият** оператор. Той просто обръща стойността наопаки!



```
a = True  
print(not a) → ✗ False
```

```
b = False  
print(not b) → ✓ True
```



Приоритет на операциите

Подобно на математиката (където умножението се извършва преди събирането), логическите операции имат строг ред на изпълнение!

1. Първо се изпълнява:

not

2. Второ се изпълнява:

and

3. Последно се изпълнява:

or

3. Педуно се изпълнява:

Финален тест: Разкодирай логиката!

```
print(not False and True)
```

Първо изпълняваме **not** (най-висок приоритет).
[not False] става True

```
True and True
```

Сега замествахме и изпълняваме **and**.

Резултат:  True

ОБОБЩЕНИЕ

Булеви стойности

`True` (1, Истина)

`False` (0, Лъжа)

Важно: Винаги с главна буква!

Сравнения

`<`, `<=`, `>`, `>=`, `==`, `!=`

Резултатът от тях винаги е `bool`.

Оператор `in`

Търси дали елемент е в списък или текст.

`"a" in "кола" → True`

Логика и Приоритет

1. `not` (обръща)
2. `and` (и двете)
3. `or` (поне едното)